МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Кафедра математического и компьютерного моделирования

Кулахсзян Сергей Грайрович

ВЫБОРКА

ЛАБОРАТОРНАЯ РАБОТА № 1

Направление подготовки 02.03.01сцт Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной формы обучения

г. Владивосток
2026

Содержание

1 Генерация выборок **2**

2 Реализация выборочных характеристик **2**

3 Сравнение расчетных характеристик **3**

4 Построение графиков **3**

4.1 Равномерное распределение 4

4.2 Распределение Бернулли 5

4.3 Биномиальное распределение 6

4.4 Нормальное распределение 8

1 Генерация выборок

Равномерное распределение $X \sim \mathrm{U}(a = -1, b = 4)$:

```
data["uniform"][100]["X"] = stats.uniform.rvs(loc=-1, scale=5, size=100, random_state=random_state)
data["uniform"][1000]["X"] = stats.uniform.rvs(loc=-1, scale=5, size=1000, random_state=random_state)
```

Распределение Бернулли $X \sim \mathrm{Bernoulli}(p = 0.25)$:

```
data["bernoulli"][100]["X"] = stats.bernoulli.rvs(p=0.25, size=100, random_state=random_state)
data["bernoulli"][1000]["X"] = stats.bernoulli.rvs(p=0.25, size=1000, random_state=random_state)
```

Биномиальное распределение $X \sim \mathrm{Bin}(n = 17, p = 0.25)$:

```
data["binom"][100]["X"] = stats.binom.rvs(p=0.25, n=17, size=100, random_state=random_state)
data["binom"][1000]["X"] = stats.binom.rvs(p=0.25, n=17, size=1000, random_state=random_state)
```

Нормальное распределение $X \sim \mathrm{N}(M = 15, \sigma = 5)$:

```
data["norm"][100]["X"] = stats.norm.rvs(loc=15, scale=5, size=100, random_state=random_state)
data["norm"][1000]["X"] = stats.norm.rvs(loc=15, scale=5, size=1000, random_state=random_state)
```

2 Реализация выборочных характеристик

Выборочное среднее:

$$\overline{x}_B = \sum_{i=1}^{n} \frac{x_i}{n}$$

```python
def mean(x: list[float]) -> float:
    return sum(x) / len(x)
```

Выборочная дисперсия:

$$D_B = \sum_{i=1}^{n} \frac{(x_i - \overline{x}_B)^2}{n}$$

```python
def dispersion(x: list[float]) -> float:
    x_mean = mean(x)
    return sum((x_i - x_mean) ** 2 for x_i in x) / len(x)
```

Выборочное стандартное отклонение:

$$\sigma_B = \sqrt{D_B}$$

```python
def standard_deviation(x: list[float]) -> float:
    return dispersion(x) ** 0.5
```

3 Сравнение расчетных характеристик

№ выборки	**Среднее**		**Дисперсия**		**Ст. отклонение**	
(распределение-размер)	моё	numpy	моя	numpy	моё	numpy
№ 1 (равномерное-100)	1.4937	1.4937	2.1570	2.1570	1.4687	1.4687
№ 2 (равномерное-1000)	1.4820	1.4820	2.0780	2.0780	1.4415	1.4415
№ 3 (Бернулли-100)	0.2700	0.2700	0.1971	0.1971	0.4440	0.4440
№ 4 (Бернулли-1000)	0.2440	0.2440	0.1845	0.1845	0.4295	0.4295
№ 5 (биномиальное-100)	4.2600	4.2600	3.6124	3.6124	1.9006	1.9006
№ 6 (биномиальное-1000)	4.2290	4.2290	3.2526	3.2526	1.8035	1.8035
№ 7 (нормальное-100)	15.9197	15.9197	30.4590	30.4590	5.5190	5.5190
№ 8 (нормальное-1000)	14.9599	14.9599	26.9451	26.9451	5.1909	5.1909

4 Построение графиков

Для начало определяется длина частотного интервала:

$$h = \frac{R}{k}$$

$R = X_{\max} - X_{\min}$
$k = 1 + 3.322 \lg n$
Тут R – размах, k – число частотных интервалов, h – длина частотного интервала

Для каждого частотного интервала $[X_{\min} + ih; X_{\min} + (i+1)h], i = \overline{0, k}$ определяется значение относительной частоты:
$w_i = \dfrac{m_i}{n}$,
где m_i – частота вхождений элементов выборки в частотный интервал $[X_{\min} + ih; X_{\min} + (i+1)h]$, n – объём выборки.

Плотность относительной частоты тогда будет равна:
$h_i = \dfrac{w_i}{h}$

Таким образом в гистограмме для каждого частотного интервала будет определён h_i

4.1 Равномерное распределение

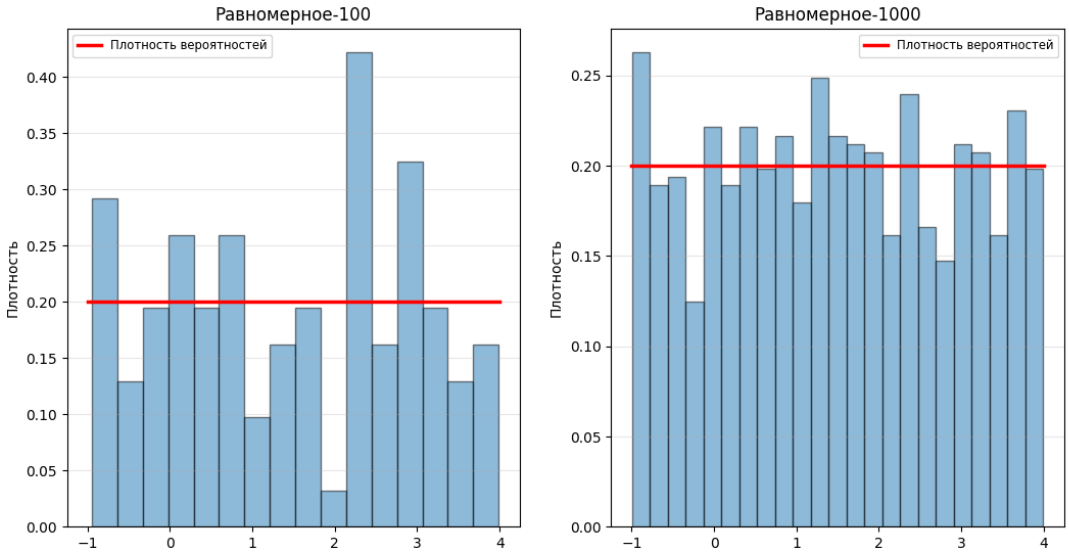
Плотность распределений для равномерного распределения имеет вид:
$f_X(x) = \dfrac{1}{b-a}$

Код построения графиков будет иметь вид:

```
def f_x(x: float) -> float:
    return 1 / (4 + 1)

fig, axes = plt.subplots(1, 2)

for col, size in enumerate([100, 1000]):
    ax = axes[col]
    sample = data["uniform"][size]["X"]

    k = int(1 + 3.322 * np.log10(len(sample)))
    h = (max(sample) - min(sample)) / k

    endpoints = np.linspace(min(sample), max(sample), k + 1)
    heights = []
    for i in range(k):
        left = endpoints[i]
        right = endpoints[i+1]
        count = ((sample >= left) & (sample < right)).sum()

        density_height = count / (size * h)
```

```
            heights.append(density_height)

        bin_centers = endpoints[:-1] + h / 2
        ax.bar(bin_centers, heights, width=h, alpha=0.5,
        edgecolor='black')

        k = np.linspace(-1, 4, 100)
        ax.plot(k, [f_x(i) for i in k], 'r-', lw=2.5,
        label='Плотность вероятностей')

        ax.set_title(f"Равномерное-{size}")
        ax.set_ylabel("Плотность")
        ax.legend(fontsize='small')
        ax.grid(axis='y', alpha=0.3)
```

Сами графики получились такими:



4.2 Распределение Бернулли

В случае распределения Бернулли случайная величина может принимать только 2 значения: 0 и 1. В связи с этим $k = 2$, то бишь будет частотный интервал для 0 и 1.

Функция вероятности распределения Бернулли имеет вид:

$$f_X(x) = \begin{cases} p, & \text{если } x = 1 \\ 1 - p, & \text{если } x = 0 \\ 0, & \text{иначе} \end{cases}$$

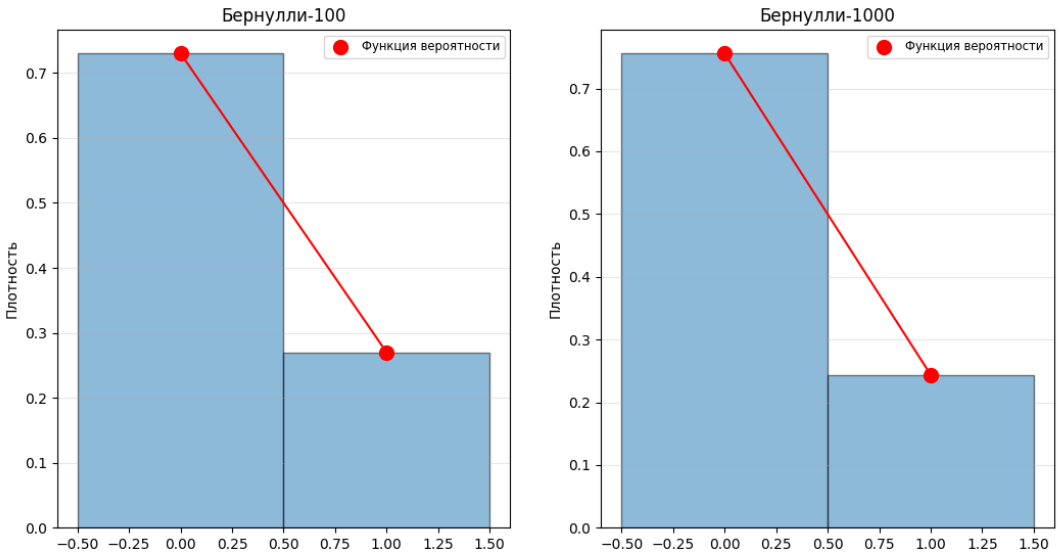
Таким образом код выглядит так:

```
fig, axes = plt.subplots(1, 2)

for col, size in enumerate([100, 1000]):
    ax = axes[col]
    sample = data["bernoulli"][size]["X"]

    m_1 = sum(sample)
    m_0 = size - m_1

    w_0 = m_0 / size
    w_1 = m_1 / size

    heights = [w_0, w_1]

    ax.bar([0, 1], heights, width=1, alpha=0.5, edgecolor='black')

    ax.scatter([0, 1], heights, s=100, zorder=5, label='Функция вероятности', color="red")
    ax.plot([0, 1], heights, color="red")

    ax.set_title(f"Бернулли-{size}")
    ax.set_ylabel("Плотность")
    ax.legend(fontsize='small')
    ax.grid(axis='y', alpha=0.3)
```

График будет выглядеть так:

4.3 Биномиальное распределение

В случае биномиального распределения интервал будет определяться целым числом от 0 до n. Функция вероятности распределения биномиального распре-



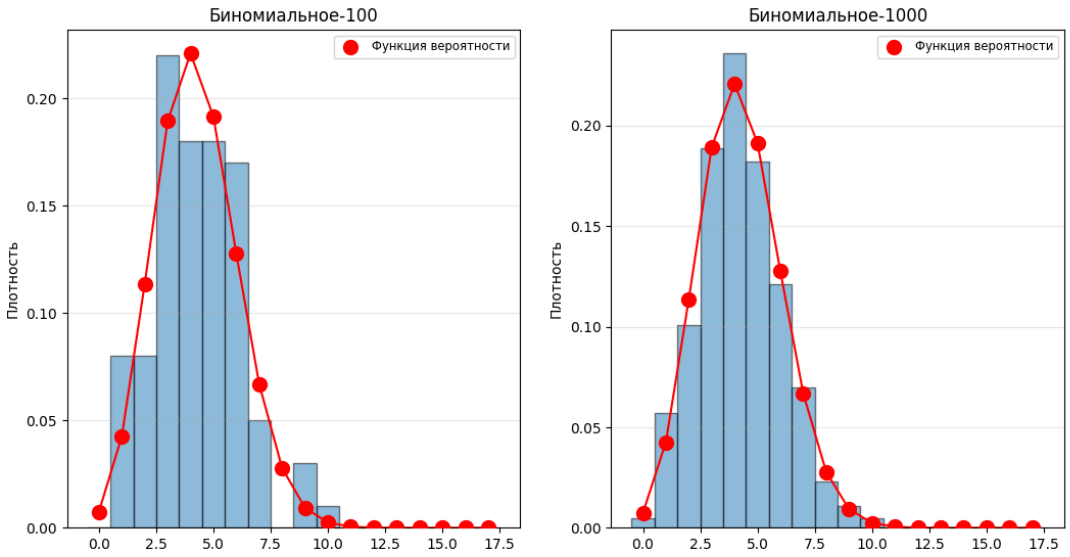
деления имеет вид:
$f_X(x) = C_n^x p^x (1-p)^{n-x}$

Таким образом код будет иметь вид:

```
def combinations(n, k) -> float:
    return math.factorial(n) / (math.factorial(k) *
    math.factorial(n - k))

def f_x(x: float) -> float:
    return combinations(17, x) * (0.25 ** x) * ((1 -
    0.25)**(17 - x))

fig, axes = plt.subplots(1, 2)

for col, size in enumerate([100, 1000]):
    ax = axes[col]
    sample = data["binom"][size]["X"]

    k = np.arange(0, 17 + 1)
    counts = [np.sum(sample == i) for i in k]

    heights = [m_i / size for m_i in counts]

    ax.bar(k, heights, width=h, alpha=0.5, edgecolor='black')
```

```python

    ax.scatter(k, [f_x(i) for i in k], s=100, zorder=5, label='Функция вероятности', color="red")
    ax.plot(k, [f_x(i) for i in k], color="red")

    ax.set_title(f"Биномиальное-{size}")
    ax.set_ylabel("Плотность")
    ax.legend(fontsize='small')
    ax.grid(axis='y', alpha=0.3)
```

Графики получились такими:



4.4 Нормальное распределение

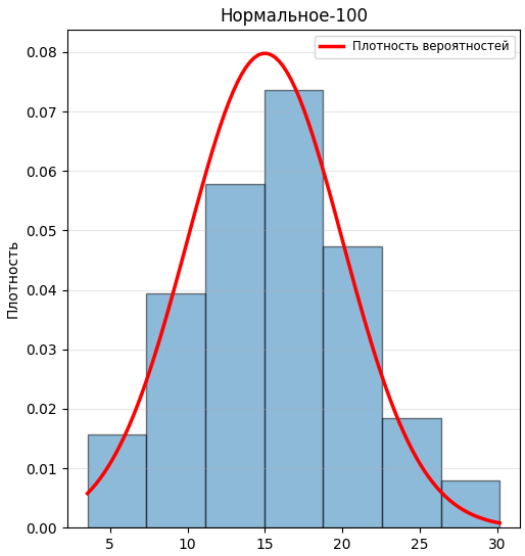
Плотность вероятности для нормального распределения имеет вид: $f_X(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-M)^2}{2\sigma^2}}$

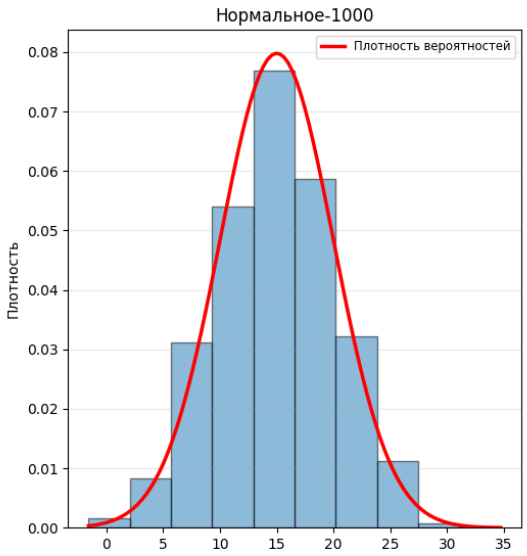
Код аналогичен равномерному распределению:

```python
def f_x(x: float) -> float:
    return 1 / (5 * np.sqrt(2 * np.pi)) * np.exp(-(x - 15) ** 2 / (2 * 5 ** 2))

```

```python
fig, axes = plt.subplots(1, 2)

for col, size in enumerate([100, 1000]):
    ax = axes[col]
    sample = data["norm"][size]["X"]

    k = int(1 + 3.322 * np.log10(len(sample)))
    h = (max(sample) - min(sample)) / k

    endpoints = np.linspace(min(sample), max(sample), k + 1)
    heights = []
    for i in range(k):
        left = endpoints[i]
        right = endpoints[i+1]
        count = ((sample >= left) & (sample < right)).sum()

        density_height = count / (size * h)
        heights.append(density_height)

    bin_centers = endpoints[:-1] + h / 2
    ax.bar(bin_centers, heights, width=h, alpha=0.5, edgecolor='black')

    k = np.linspace(min(sample), max(sample), 100)
    ax.plot(k, [f_x(i) for i in k], 'r-', lw=2.5, label='Плотность вероятностей')

    ax.set_title(f"Нормальное-{size}")
    ax.set_ylabel("Плотность")
    ax.legend(fontsize='small')
    ax.grid(axis='y', alpha=0.3)
```

График вышел таким:


Нормальное-100
Плотность вероятностей
Плотность
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0.00
5
10
15
20
25
30


Нормальное-1000
Плотность вероятностей
Плотность
0.08
0.07
0.06
0.05
0.04
0.03
0.02
0.01
0.00
0
5
10
15
20
25
30
35